Capstone Project: Capstone 1

Global Fleet Intelligence

A take-away project after the first two days

KavinSchool

What This Capstone Is

- A take-home project you complete after Days 1 and 2
- Built around Global, a global automotive and fleet services company
- Three exercises that grow in complexity
- Uses only Day 1 and Day 2 skills: API calls, prompts, structured output, parameters, evaluation

> No RAG, no chatbots, no production tooling. Those arrive in Days 3 and 4.

The Business Problem

Fleet operations generate unstructured text faster than people can read it:

- Drivers report vehicle problems in plain language
- Repair shops submit work orders in inconsistent formats
- Dispatchers and telematics raise alerts that need triage

Each exercise turns one text stream into structured, actionable data using prompt engineering alone.

Skills You Will Apply

Day	Skills exercised
Day-1	API calls, roles, capabilities and limits, input prep, cost estimation
Day-2	Building blocks, few-shot, chain-of-thought, role prompting, structured output, parameters, evaluation

The Three Exercises

#	Exercise	Complexity
1	Driver Issue Classifier	Low to mid
2	Work Order Parser and Cost Checker	Mid
3	Fleet Service Advisor (mini project)	High

> Do them in order. Each one reuses prompting skills from the last.

Exercise 1

Driver Issue Classifier

KavinSchool

Exercise 1: The Task

A driver texts a vehicle problem. Classify it instantly so safety-critical issues jump the queue.

Output: JSON with `category`, `severity`, `drivable`, `recommended_action`, `summary`.

You practice:

- Role-based system message with explicit rules
- Forcing valid JSON, `temperature=0.0`
- Few-shot examples to fix severity boundaries

Exercise 1: Prompt Sketch

```
"""You are a fleet service triage assistant for Global.
Respond with ONLY JSON:
{"category": "...", "severity": "SAFETY_CRITICAL|HIGH|MEDIUM|LOW",
 "drivable": true/false, "recommended_action": "...", "summary": "..."}

SAFETY_CRITICAL: brakes failing, steering loss, smoke, blowout.
If unsafe to drive, set drivable=false and action=TOW_NOW.
Do not invent details not in the report."""
```

> **Done when:** all eight sample reports classify sensibly and parse on the first try.

Exercise 2

Work Order Parser and Cost Checker

KavinSchool

Exercise 2: The Task

Repair shops send messy work orders. Parse them into line items, then judge whether the total is reasonable.

Output: JSON line items + a `cost_assessment` of WITHIN_RANGE, HIGH, or SUSPICIOUS with a reason.

You practice:

- Parsing inconsistent free-text input
- Few-shot to anchor the output shape
- Chain-of-thought so the cost call is reasoned
- Token and cost estimation (Day 1)

Exercise 2: The Reasoning Step

```
"""Reason step by step about whether the total is
reasonable for the work described, then set cost_assessment:
- WITHIN_RANGE: normal for this service and parts
- HIGH: above typical but plausible
- SUSPICIOUS: total does not match line items, or
  charges look vague or invented

The sum of line items should equal total. If not -> SUSPICIOUS."""
```

Done when: parsed totals reconcile, assessments are defensible, per-call cost is printed.

Exercise 3

Fleet Service Advisor (mini project)

KavinSchool

Exercise 3: The Task

One intake assistant that triages a report, drafts messages, and proves itself with an evaluation harness.

1. **Triage** the report into structured JSON
2. **Generate** a driver notification and a dispatcher action line
3. **Build** a golden dataset of labeled cases
4. **Evaluate** with accuracy plus LLM-as-judge
5. **Version** the prompt: run two variants, compare, decide

Exercise 3: The Evaluation Loop

This is the Day 2 iteration loop in miniature.

Method	What it measures
Field accuracy	Classification vs golden labels (deterministic)
LLM-as-judge	Driver message on clarity, tone, faithfulness
Version compare	V1 vs V2 prompt accuracy and judge scores

> **Done when:** the harness prints accuracy per version, a judge score, and a justified winner.

Exercise 3: Tone via Temperature

```
# Classification and parsing: deterministic
triage(report, prompt)              # temperature=0.0

# Driver-facing message: a little warmth
generate_messages(report, triage) # temperature=0.4

# The judge: deterministic again
judge_message(report, triage, msg) # temperature=0.0
```

> Raise temperature only where natural language helps. Keep judgments and structure at 0.0.

Self-Assessment Rubric

Criterion	Points	Good looks like
Prompt engineering	30	Clear role, rules, few-shot, right parameters
Structured output	20	Valid JSON every time, graceful on failure
Reasoning quality	15	Chain-of-thought where judgment is needed
Evaluation (Ex. 3)	25	Golden set, accuracy, judge, version compare
Code clarity	10	Readable, runs from a clean checkout

> Not graded. Use it to check completeness.

Working Tips

- Start with the simplest prompt that works, then iterate
- Keep `temperature=0.0` for classification and parsing
- When JSON parsing fails, read the raw output before changing the prompt
- Watch token usage with `get_completion_full(...).usage`
- Never let the model invent a VIN, a price, or a safety judgment

Getting Started

This capstone is located in the companion repository:

`https://github.com/kpassoubady/gen-ai-companion`

```
# From the companion repository root
cd capstone-1/start
python3 exercise1_issue_classifier.py
python3 exercise2_work_order_parser.py
python3 exercise3_fleet_service_advisor.py
```

Files: starter code in `capstone-1/start/`, solutions in `capstone-1/solution/`. Full brief in `capstone-1/README.md`.

> Implement one TODO at a time and run after each.

Build It

Turn fleet text into structured, evaluated data

Everything you need comes from Days 1 and 2.

KavinSchool